

TASK 4 A & B

1. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
gopal@PC: /tmp

[1]+  Done                  sleep 30
gopal@PC:/tmp$ cat /proc/12345/status | grep State
cat: /proc/12345/status: No such file or directory
gopal@PC:/tmp$ cd /tmp
gopal@PC:/tmp$ printf '#include <stdio.h>\n#include <stdlib.h>\n#include <unistd.h>\n#include <sys/wait.h>\n\nint main() {\n    int n = 3;\n    pid_t pids[3];\n\n    for (int i = 0; i < n; i++) {\n        pid_t pid = fork();\n        if (pid == 0) {\n            printf("Child %d: PID=%d\n", i, getpid());\n            sleep(i + 1);\n            exit(i);\n        } else {\n            pids[i] = pid;\n        }\n    }\n\n    for (int i = 0; i < n; i++) {\n        int status;\n        waitpid(pids[i], &status, 0);\n        printf("Reaped child PID=%d, exit code=%d\n", pids[i], WEXITSTATUS(status));\n    }\n\n    return 0;\n}' > task4a.c
gopal@PC:/tmp$ cat task4a.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int n = 3;
    pid_t pids[3];

    for (int i = 0; i < n; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            printf("Child %d: PID=%d\n", i, getpid());
            sleep(i + 1);
            exit(i);
        } else {
            pids[i] = pid;
        }
    }

    for (int i = 0; i < n; i++) {
        int status;
        waitpid(pids[i], &status, 0);
        printf("Reaped child PID=%d, exit code=%d\n", pids[i], WEXITSTATUS(status));
    }

    return 0;
}
gopal@PC:/tmp$
```

OUTPUT :

```
gopal@PC: /tmp
gopal@PC:/tmp$ cat task4a.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int n = 3;
    pid_t pids[3];

    for (int i = 0; i < n; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            printf("Child %d: PID=%d\n", i, getpid());
            sleep(i + 1);
            exit(i);
        } else {
            pids[i] = pid;
        }
    }

    for (int i = 0; i < n; i++) {
        int status;
        waitpid(pids[i], &status, 0);
        printf("Reaped child PID=%d, exit code=%d\n", pids[i],
            WEXITSTATUS(status));
    }

    return 0;
}
gopal@PC:/tmp$ gcc task4a.c -o task4a
gopal@PC:/tmp$ ./task4a
Child 0: PID=1352
Child 1: PID=1353
Child 2: PID=1354
Reaped child PID=1352, exit code=0
Reaped child PID=1353, exit code=1
Reaped child PID=1354, exit code=2
gopal@PC:/tmp$
```

4B:

```
gopal@PC: /tmp
[1]+  Done                  sleep 30
gopal@PC:/tmp$ cat /proc/12345/status | grep State
cat: /proc/12345/status: No such file or directory
gopal@PC:/tmp$ cd /tmp
gopal@PC:/tmp$ printf '#include <stdio.h>\n#include <stdlib.h>\n#include <unistd.h>\n#include <sys/wait.h>\n\nint main() {\n    int n = 3;\n    pid_t pids[3];\n\n    for (int i = 0; i < n; i++) {\n        pid_t\n        pid = fork();\n        if (pid == 0) {\n            printf("Child    %d:    PID=%d\n", i, getpid());\n            sleep(i + 1);\n            exit(i);\n        } else {\n            pids[i] = pid;\n        }\n    }\n\n    for (int i = 0; i < n; i++) {\n        int status;\n        waitpid(pids[i], &status, 0);\n        printf\n        ("Reaped child PID=%d, exit code=%d\n", pids[i], WEXITSTATUS(status));\n    }\n    return 0;\n}\n' > task4a.c
gopal@PC:/tmp$ cat task4a.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    int n = 3;
    pid_t pids[3];

    for (int i = 0; i < n; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            printf("Child %d: PID=%d\n", i, getpid());
            sleep(i + 1);
            exit(i);
        } else {
            pids[i] = pid;
        }
    }

    for (int i = 0; i < n; i++) {
        int status;
        waitpid(pids[i], &status, 0);
        printf("Reaped child PID=%d, exit code=%d\n", pids[i], WEXITSTATUS(status));
    }
    return 0;
}
gopal@PC:/tmp$ gcc task4a.c -o task4a
gopal@PC:/tmp$ ./task4a
Child 0: PID=1352
Child 1: PID=1353
Child 2: PID=1354
Reaped child PID=1352, exit code=0
Reaped child PID=1353, exit code=1
Reaped child PID=1354, exit code=2
gopal@PC:/tmp$
```